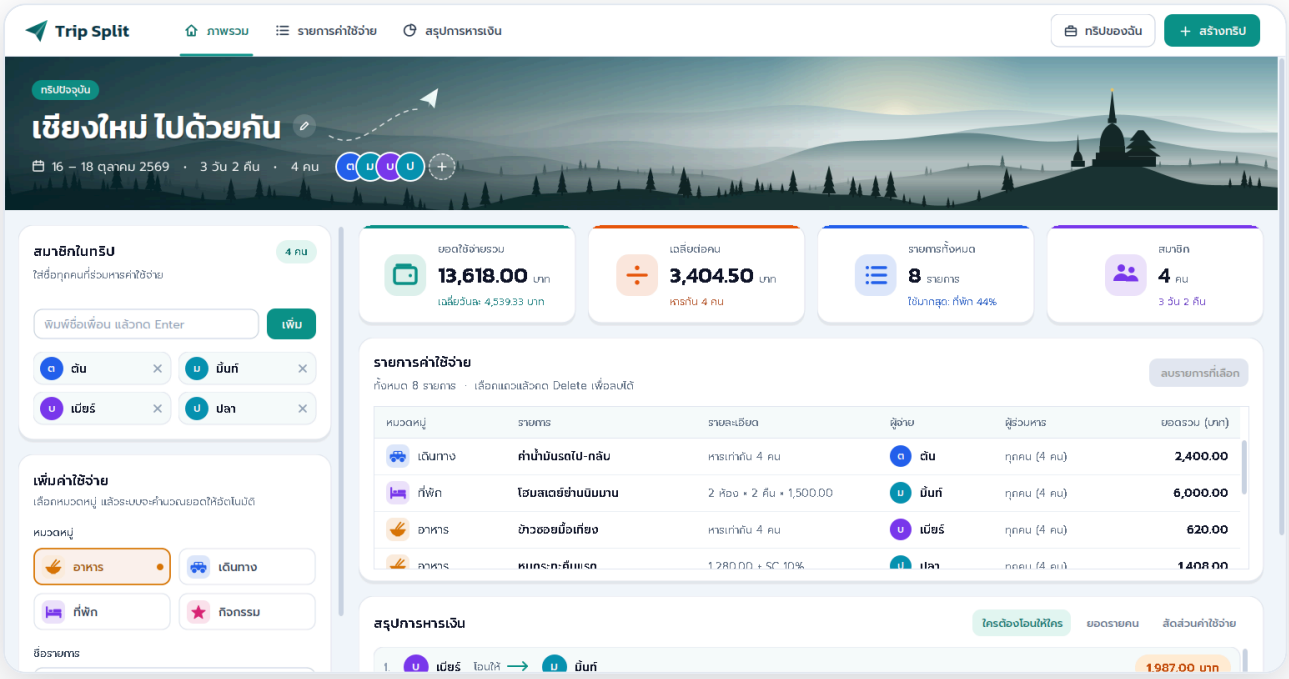
WORKSHOP · JAVA SWING GUI · OOP

สรุปโปรเจกต์ (Project Summary)

Trip Split

ระบบจัดทริปเที่ยวและหารค่าใช้จ่าย

บันทึกค่าใช้จ่ายระหว่างทริป แล้วคำนวณให้อัตโนมัติว่าใครต้องโอนให้ใคร เท่าไร พัฒนาด้วย Java Swing ตามหลัก OOP ครบ 4 ข้อ และแยกส่วน Model (ข้อมูล + การคำนวณ) ออกจาก UI (หน้าจอ) อย่างชัดเจน



4 / 4	39	22 / 22	0
หลัก OOP ที่ใช้ครบ	ไฟล์ Java (Model 11 · UI 25)	Test Case ผ่านทั้งหมด	ไลบรารีภายนอก (Java ล้วน)

ฐานทรัพย์ หาญเชิงชัย

รหัสนักศึกษา 66040233151 · สาขาเทคโนโลยีสารสนเทศ

มหาวิทยาลัยราชภัฏอุดรธานี

Java 17+ · Swing

ตุลาคม 2569

สารบัญ

1	ภาพรวมของระบบ	7	อัลกอริทึมคำนวณการโอนเงิน
2	ความสามารถของระบบ	8	การตรวจสอบข้อมูลและเทคนิคด้านหน้าจอ
3	ขั้นตอนการใช้งาน	9	โครงสร้างโปรเจกต์: โค้ดส่วนไหนทำอะไร
4	หลักการที่ใช้: แยก Model / UI	10	ลำดับการทำงานเมื่อกดบันทึก
5	หลักการ OOP 4 ข้อ	11	การทดสอบ (Test Case)
6	Class Diagram และ Polymorphism	12	วิธีรันโปรแกรมและสรุป

1 ภาพรวมของระบบ

ปัญหา: เวลาไปเที่ยวกับเพื่อน แต่ละคนผลัดกันจ่ายค่าน้ำมัน ค่าที่พัก ค่าอาหาร ค่าตั๋ว พอจบทริปมักสับสนว่าใครจ่ายไปเท่าไร และใครต้องโอนคืนให้ใคร บางรายการก็ไม่ได้หารกันทุกคน เช่น บางคนไม่ได้เข้าร่วมกิจกรรม

แนวคิด: ให้ทุกคนบันทึกค่าใช้จ่ายลงโปรแกรมทันทีที่จ่าย โดยระบุว่า *ใครจ่าย* และ *หารกับใคร* แต่ละประเภทค่าใช้จ่ายมีสูตรคำนวณของตัวเอง เมื่อจบทริประบบจะสรุปยอดของแต่ละคน และบอกรายการโอนเงินที่**โอนน้อยครั้งที่สุด**ก็เคลียร์ครบทุกคน

ผลลัพธ์: โปรแกรม Java Swing แบบหน้าต่างเดียว (Dashboard) ภาษาไทยทั้งหมด หน้าตาทันสมัย ใช้หลัก OOP ครบ 4 ข้อ (Encapsulation, Inheritance, Polymorphism, Abstraction) และความสัมพันธ์แบบ Has-a โดยแยก Logic ทั้งหมดไว้ในแพ็กเกจ `model` ส่วนแพ็กเกจ `ui` มีหน้าที่แสดงผลอย่างเดียว

> **ตัวอย่างในเอกสารนี้** ใช้ "ข้อมูลตัวอย่าง" ในโปรแกรม: ทริป **เชียงใหม่ ไปด้วยกัน** 16–18 ตุลาคม 2569 (3 วัน 2 คืน) สมาชิก 4 คน (ต้น มิ้นท์ เบียร์ ปลา) มีค่าใช้จ่าย 8 รายการ รวม **13,618.00 บาท**

ส่วนประกอบของหน้าจอ

ส่วน	รายละเอียด
แถบเมนูด้านบน	โลโก้, แท็บ ภาพรวม / รายการค่าใช้จ่าย / สรุปการหารเงิน, ปุ่ม "ทริปของฉัน" และ "สร้างทริป"
แบนเนอร์ทริป	ภาพวิวภูเขา (วาดด้วยโค้ด) แสดงชื่อทริป วันที่ จำนวนวัน จำนวนคน และรูปสมาชิก
คอลัมน์ซ้าย	การ์ดสมาชิกในทริป และฟอร์มเพิ่มค่าใช้จ่าย
การ์ดสรุป 4 ใบ	ยอดใช้จ่ายรวม · เฉลี่ยต่อคน · รายการทั้งหมด · สมาชิก (มีบรรทัดสรุปใต้ตัวเลข)
ตารางค่าใช้จ่าย	หมวดหมู่ รายการ รายละเอียดสูตร ผู้จ่าย ผู้ร่วมหาร ยอดรวม พร้อมปุ่มลบ
การ์ดสรุปการหารเงิน	3 แท็บ: ใครต้องโอนให้ใคร / ยอดรายคน / สัดส่วนค่าใช้จ่าย (กราฟโดนัท)

2 ความสามารถของระบบ

1 จัดการทริป
สร้างทริปใหม่ ตั้งชื่อ เลือกวันเดินทาง–วันกลับ จากปฏิทิน (ปี พ.ศ.) ระบบนับจำนวนวัน/คืนให้

2 จัดการสมาชิก
เพิ่ม/ลบสมาชิกได้สูงสุด 20 คน ห้ามชื่อซ้ำ และลบไม่ได้ถ้ายังมีรายการที่เกี่ยวข้อง

3 ค่าใช้จ่าย 4 หมวด
อาหาร · เดินทาง · ที่พัก · กิจกรรม แต่ละหมวดมีช่องกรอกและสูตรคำนวณต่างกัน

4 เลือกผู้จ่าย / ผู้ร่วมหาร
ระบุว่าใครจ่าย และหารกับใครบ้าง (ไม่ต้องทุกคน) กิจกรรมกรอกราคาแยกรายคนได้

5 ดูยอดแบบสด
ขณะกรอก ระบบแสดงยอดรวมและเฉลี่ยต่อคนทันที ก่อนกดบันทึก

6 สรุปใครต้องโอนให้ใคร
คำนวณยอดสุทธิของแต่ละคน แล้วจับคู่การโอนให้น้อยครั้งที่สุด

7 ยอดรายคน + กราฟ
ตารางจ่ายไปแล้ว / ส่วนที่ต้องจ่ายจริง / สถานะ และกราฟโดนัทสัดส่วนแต่ละหมวด

8 คัดลอกสรุปไปแชท
กดปุ่มเดียว ได้ข้อความสรุปยอดและรายการโอน วางในแชทกลุ่มได้ทันที

9 ตรวจสอบข้อมูล
แจ้งเตือนภาษาไทยด้วย JOptionPane เมื่อกรอกไม่ครบหรือไม่ถูกต้อง

10 ข้อมูลตัวอย่าง
โหลดทริปตัวอย่างได้ทันทีเพื่อทดลองหรือนำเสนอ (หรือรันด้วย `--demo`)

สูตรของค่าใช้จ่ายแต่ละหมวด

หมวด	ช่องที่กรอก	ยอดรวม (getTotal)	ส่วนของแต่ละคน (getShareFor)
อาหาร	ค่าอาหาร, Service Charge	ค่าอาหาร × 1.10 (ถ้าติ๊ก Service Charge 10%)	หารเท่ากันทุกคนที่ร่วมหาร
เดินทาง	จำนวนเงิน	จำนวนเงินที่จ่าย	หารเท่ากันทุกคนที่ร่วมหาร
ที่พัก	ราคา/คืน, จำนวนคืน, ห้อง	ราคาต่อคืน × จำนวนคืน × จำนวนห้อง	หารเท่ากันทุกคนที่ร่วมหาร
กิจกรรม	ราคาของแต่ละคน	ผลรวมราคาของทุกคน	เท่าราคาของตัวเอง (คนที่ไม่ไปกรอก 0)

3 ขั้นตอนการใช้งาน

1. **สร้างทริป**
   กด **+ สร้างทริป** ที่มุมขวาบน ตั้งชื่อ และเลือกวันเดินทาง–วันกลับจากปฏิทิน (แก้ไขภายหลังได้โดยคลิกชื่อทริปบนแบนเนอร์ หรือเมนู "ทริปของฉัน")

2. **เพิ่มสมาชิก**
   พิมพ์ชื่อเพื่อนในการ์ด "สมาชิกในทริป" แล้วกด Enter หรือปุ่ม **เพิ่ม** (กดปุ่ม + บนแบนเนอร์ก็ได้)

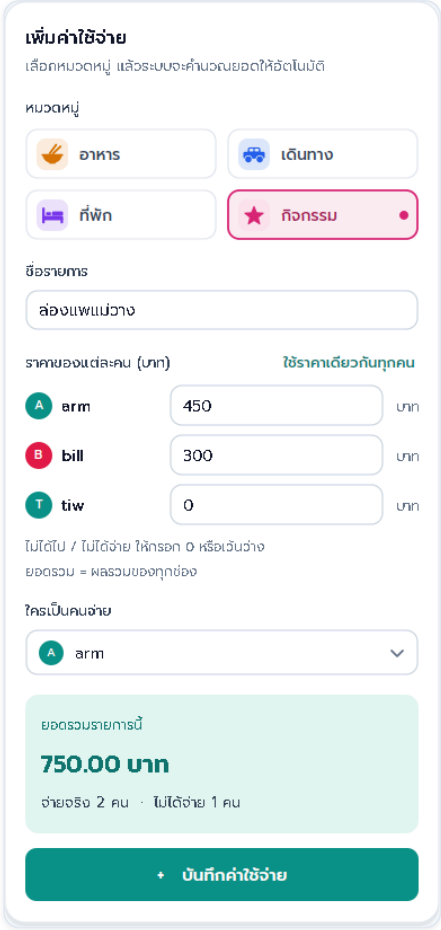
3. **เลือกหมวดและกรอกค่าใช้จ่าย**
   เลือกหมวด อาหาร / เดินทาง / ที่พัก / กิจกรรม ช่องกรอกจะเปลี่ยนตามหมวด เช่น ที่พักกรอกราคาต่อคืน จำนวนคืน และจำนวนห้อง ส่วนกิจกรรมกรอกราคาแยกรายคน (มีปุ่ม "ใช้ราคาเดียวกันทุกคน")

4. **เลือกผู้จ่ายและผู้ร่วมหาร**
   เลือกคนที่ควักจ่าย และติ๊กคนที่ร่วมหาร กล่องสีเขียวจะแสดงยอดรวมและเฉลี่ยต่อคนให้ดูก่อน

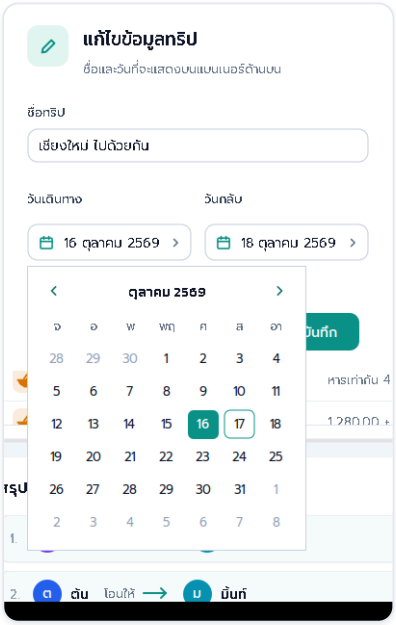
5. **กดบันทึกค่าใช้จ่าย**
   รายการจะขึ้นในตาราง การ์ดสรุปและการหารเงินคำนวณใหม่ทันที ถ้าข้อมูลไม่ถูกต้องจะมีหน้าต่างแจ้งเตือน

6. **ดูสรุปและโอนเงิน**
   ไปที่แท็บ **สรุปการหารเงิน** ดูว่าใครต้องโอนให้ใคร แล้วกด **คัดลอกสรุป** ไปวางในแชทกลุ่ม

7. **ลบรายการ (ถ้ากรอกผิด)**
   เลือกแถวในตาราง แล้วกดปุ่ม "ลบรายการที่เลือก" หรือปุ่ม Delete บนคีย์บอร์ด



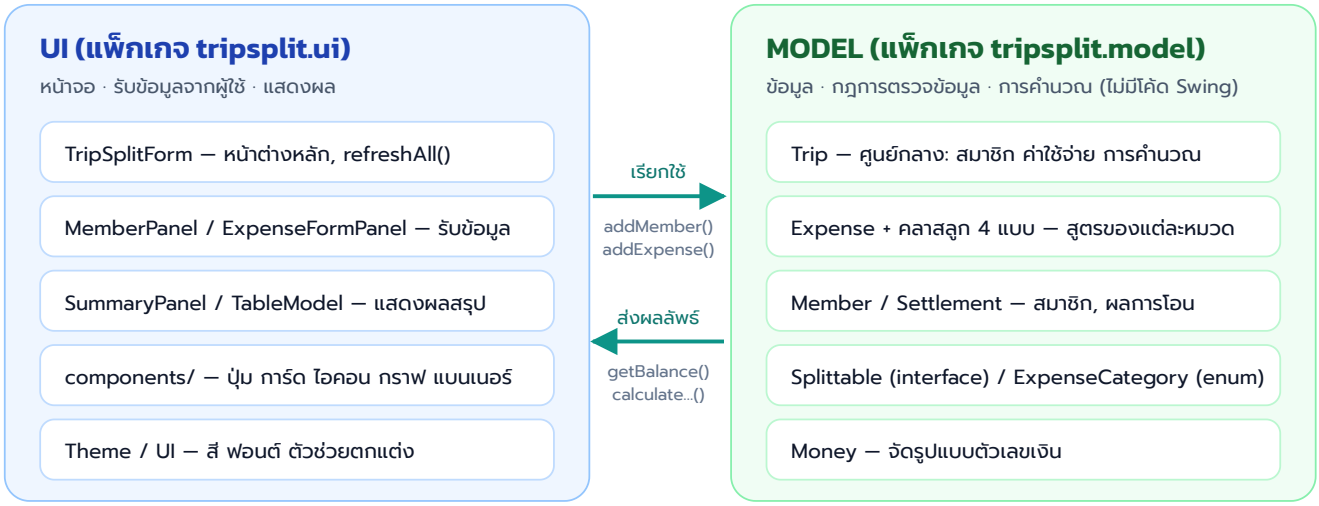
ฟอร์มหมวดกิจกรรม: กรอกราคาแยกรายคน คนที่ไม่ไปกรอก 0



หน้าต่างแก้ไขทริปพร้อมปฏิทิน (ปี พ.ศ.)

4 หลักการที่ใช้: แยก Model และ UI

โปรแกรมแบ่งโค้ดเป็น 2 ส่วนหลักตามที่โจทย์กำหนด ทำให้แก้หน้าจอได้โดยไม่กระทบการคำนวณ และทดสอบการคำนวณได้โดยไม่ต้องเปิดหน้าจอ (ไฟล์ `TripModelTest` ทดสอบ Model ล้วน ๆ)



เรื่อง	Model	UI
หน้าที่	เก็บข้อมูล ตรวจความถูกต้อง คำนวณยอด	จัดวางหน้าจอ รับค่าจากผู้ใช้ แสดงผล
ใช้ Swing?	ไม่ใช้เลย (import เฉพาะ java.util / java.time)	ใช้ JFrame, JPanel, JTable, Graphics2D
การตรวจข้อมูล	โยน `IllegalArgumentException` พร้อมข้อความภาษาไทย	จับ Exception แล้วแสดงด้วย `JOptionPane`
เมื่อข้อมูลเปลี่ยน	—	เรียก `refreshAll()` วาดทุกส่วนใหม่จาก Trip

ตัวอย่างการแยกหน้าที่: เมื่อกดเพิ่มสมาชิกชื่อซ้ำ `MemberPanel` (UI) จะไม่ตรวจเอง แต่เรียก `trip.addMember(name)` แล้ว `Trip` (Model) เป็นคนตรวจและโยนข้อความ "มีชื่อ … อยู่ในทริปแล้ว" UI แค่จับข้อความนั้นไปแสดงในหน้าต่างแจ้งเตือน

5 หลักการ OOP 4 ข้อ

Encapsulation · การห่อหุ้มข้อมูล

field ทุกตัวเป็น `private` เข้าถึงผ่าน getter ตรวจข้อมูลใน constructor และคืน list แบบแก้ไม่ได้ ภายนอกจึงเปลี่ยนข้อมูลผิดกฎไม่ได้

ใช้ที่: Member, Expense, Trip, Settlement, Stepper

```
private final List<Member> members = new
ArrayList<>();

public List<Member> getMembers() {
    return
Collections.unmodifiableList(members);
}
```

Inheritance · การสืบทอด

คลาสลูก 4 แบบ `extends Expense` ได้ชื่อรายการ ผู้จ่าย ผู้ร่วมหาร และการตรวจข้อมูลจากคลาสแม่ โดยส่งค่าร่วมผ่าน `super(...)`

ใช้ที่: Food/Transport/Hotel/ActivityExpense, Renderer, Component ที่ออกแบบเอง

```
public class HotelExpense extends Expense {
    public HotelExpense(String title, Member
payer, ...) {
        super(title, payer, participants);
        ...
```

Polymorphism · การพ้องรูป

`Trip` เรียกเมธอดเดียว `e.getTotal()` กับทุกรายการ แต่แต่ละคลาสลูกคิดคนละสูตร และ `ActivityExpense` override `getShareFor()` ให้แต่ละคนจ่ายเท่าราคาของตัวเอง

ใช้ที่: Trip.getTotalSpending(), getTotalShareOf(), ตารางค่าใช้จ่าย

```
for (Expense e : expenses) {
    total += e.getTotal(); // คนละสูตรตามคลาส
ลูก
}
```

Abstraction · นามธรรม

interface `Splittable` กำหนดว่า "สิ่งที่หารเงินได้" ต้องทำอะไรได้บ้าง และ abstract class `Expense` บังคับให้คลาสลูกเขียน `getTotal()`, `getDetail()`, `getCategory()`

ใช้ที่: Splittable, Expense

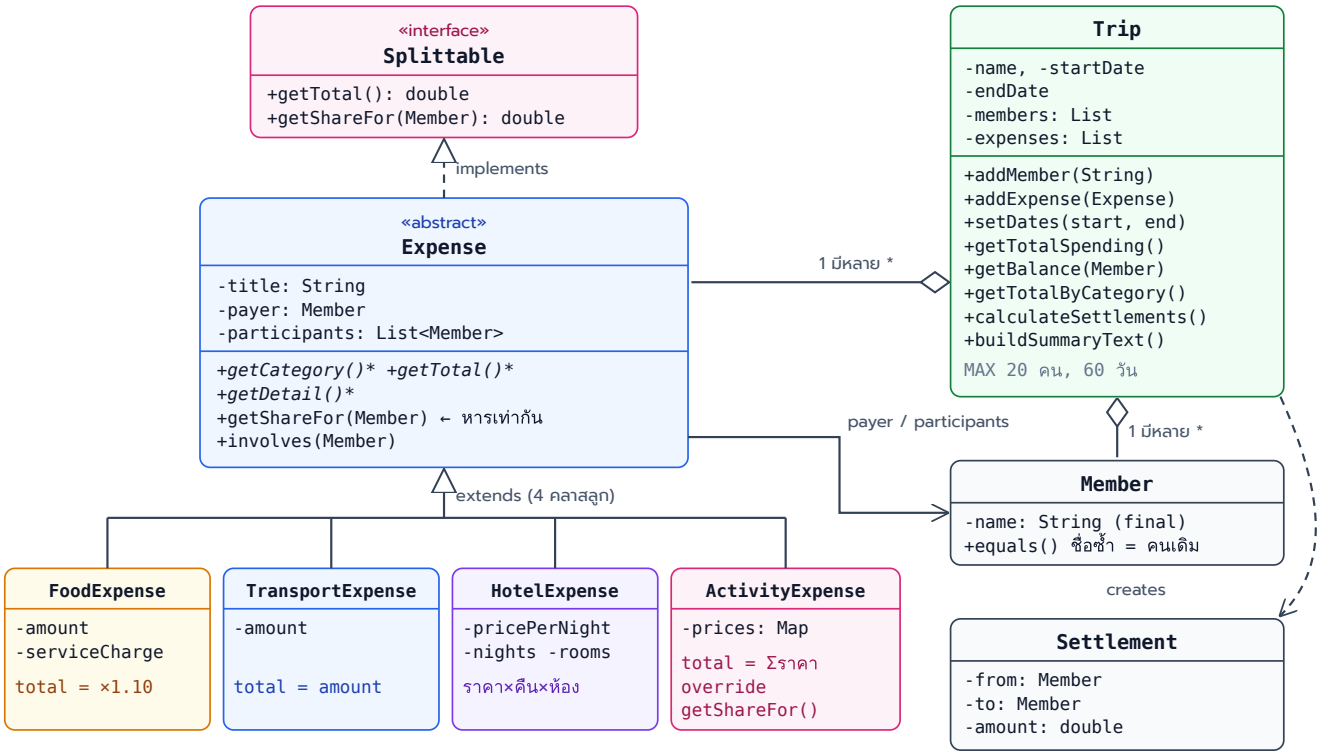
```
public abstract class Expense implements
Splittable {
    public abstract double getTotal();
    public abstract String getDetail();
```

ความสัมพันธ์ระหว่างคลาส (Has-a)

ความสัมพันธ์	ชนิด	ความหมาย
Trip → Member	Has-many (1 : *)	ทริปหนึ่งมีสมาชิกหลายคน (สูงสุด 20)
Trip → Expense	Has-many (1 : *)	ทริปหนึ่งมีค่าใช้จ่ายหลายรายการ
Expense → Member (payer)	Has-a (1 : 1)	แต่ละรายการมีผู้จ่าย 1 คน
Expense → Member (participants)	Has-many (1 : 1..*)	แต่ละรายการมีผู้ร่วมหารอย่างน้อย 1 คน
Trip → Settlement	สร้าง (creates)	`calculateSettlements()` สร้างรายการโอนเงิน

นอกจากนี้ยังใช้ **Constructor Overloading** (RoundedPanel, RoundedBorder), **enum** (ExpenseCategory, ModernButton.Style), **Inner class** (CalendarPanel, NavTab) และ **Callback** ผ่าน Runnable/Consumer เพื่อแจ้งหน้าหลักเมื่อข้อมูลเปลี่ยน

6 Class Diagram และ Polymorphism



Polymorphism ในการทำงานจริง (ข้อมูลตัวอย่าง)

ตารางนี้เกิดจากการเรียก `getDetail()` และ `getTotal()` ของแต่ละรายการ ด้วยคำสั่งเดียวกัน แต่ได้ผลตามสูตรของคลาสลูกแต่ละตัว

รายการ	คลาส	ผู้จ่าย	getDetail()	getTotal()
ค่าน้ำมันรถไป-กลับ	Transport	ต้น	หารเท่ากัน 4 คน	2,400.00
โฮมสเตย์ย่านนิมมาน	Hotel	มิ้นท์	2 ห้อง × 2 คืน × 1,500.00	6,000.00
ข้าวซอยมื้อเที่ยง	Food	เบียร์	หารเท่ากัน 4 คน	620.00
หมูกระทะคืนแรก	Food	ปลา	1,280.00 + SC 10%	1,408.00
ตั๋วเข้าสวนดอกไม้	Activity	ต้น	ต้น 150 · มิ้นท์ 150 · ปลา 150 (เบียร์ไม่ไป)	450.00
เหมารถแดงขึ้นดอยสุเทพ	Transport	เบียร์	หารเท่ากัน 4 คน	600.00
คาเฟ่วิวดอย	Food	มิ้นท์	หารเท่ากัน 2 คน (มิ้นท์, ปลา)	340.00
ล่องแพแม่วาง	Activity	ปลา	คนละ 450 × 4 คน	1,800.00
รวมทั้งทริป (`trip.getTotalSpending()`)				**13,618.00**

7 อัลกอริทึมคำนวณการโอนเงิน

หัวใจของระบบอยู่ที่เมธอด `Trip.calculateSettlements()` ทำงาน 3 ขั้น:

1. **หายอดสุทธิของแต่ละคน**: `getBalance(m) = getTotalPaidBy(m) − getTotalShareOf(m)` ถ้าเป็น**บวก** = ต้องได้เงินคืน ถ้าเป็น**ลบ** = ต้องจ่ายเพิ่ม (ผลรวมยอดสุทธิของทุกคน = 0 เสมอ)
2. **แยกเจ้าหนี้กับลูกหนี้** แล้วเรียงจากยอดมากไปน้อย (ตัวช่วย `insertSorted()`)
3. **จับคู่แบบ Greedy**: ให้ลูกหนี้ยอดมากสุดโอนให้เจ้าหนี้ยอดมากสุด เท่ากับยอดที่น้อยกว่าของสองฝั่ง คนที่ยอดเหลือ 0 ก็ขยับไปคนถัดไป ทำซ้ำจนครบ ผลคือจำนวนครั้งที่โอน**ไม่เกิน (จำนวนสมาชิก − 1)**

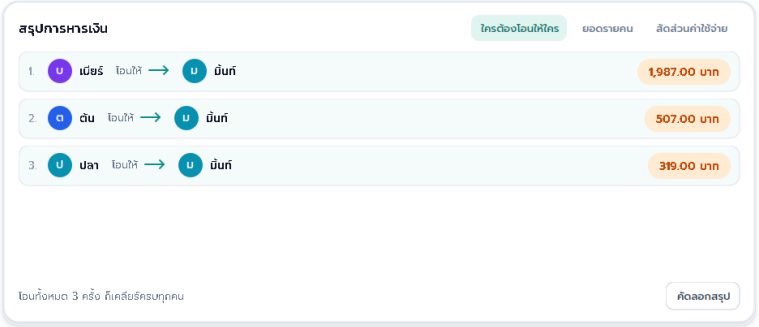
> **กันทศนิยมคลาดเคลื่อน:** ระหว่างจับคู่ ระบบแปลงเงินเป็นหน่วย **สตางค์** (`long`) เช่น 100 บาทหาร 3 คน ได้คนละ 33.33 บาท โดยไม่มีเศษ 0.0000001 ค้างจนเกิดรายการโอน 0.00 บาท (ทดสอบไว้ใน TC17)

ตัวอย่างการคำนวณ (ข้อมูลตัวอย่าง)

สมาชิก	จ่ายไปแล้ว	ส่วนที่ต้องจ่ายจริง	ยอดสุทธิ	สถานะ
ต้น	2,850.00	3,357.00	-507.00	ต้องจ่ายเพิ่ม
มิ้นท์	6,340.00	3,527.00	+2,813.00	ได้เงินคืน
เบียร์	1,220.00	3,207.00	-1,987.00	ต้องจ่ายเพิ่ม
ปลา	3,208.00	3,527.00	-319.00	ต้องจ่ายเพิ่ม
รวม	**13,618.00**	**13,618.00**	**0.00**	

เจ้าหนี้มีคนเดียวคือ มิ้นท์ (+2,813) ลูกหนี้เรียงจากมากไปน้อย: เบียร์ 1,987 → ต้น 507 → ปลา 319 จึงได้ผลลัพธ์ **โอน 3 ครั้งก็เคลียร์ครบ**

#	ผู้โอน	ผู้รับ	จำนวนเงิน	ยอดคงเหลือของมิ้นท์
1	เบียร์	มิ้นท์	1,987.00	2,813 - 1,987 = 826
2	ต้น	มิ้นท์	507.00	826 - 507 = 319
3	ปลา	มิ้นท์	319.00	319 - 319 = 0 ✓



แท็บ "ใครต้องโอนให้ใคร" ในโปรแกรม แสดงผลตรงกับการคำนวณด้านบน

8 การตรวจสอบข้อมูลและเทคนิคด้านหน้าจอ

การตรวจสอบข้อมูล (Validation)

กรณี	ข้อความแจ้งเตือน	ตรวจที่
ชื่อสมาชิกว่าง / ยาวเกิน 30 ตัว	กรุณากรอกชื่อสมาชิก / ชื่อสมาชิกยาวได้ไม่เกิน 30 ตัวอักษร	Member
ชื่อสมาชิกซ้ำ / เกิน 20 คน	มีชื่อ "..." อยู่ในทริปแล้ว / เพิ่มสมาชิกได้สูงสุด 20 คน	Trip.addMember
ลบสมาชิกที่มีรายการเกี่ยวข้อง	ไม่สามารถลบ "..." ได้ เพราะมีรายการค่าใช้จ่ายที่เกี่ยวข้อง	Trip.removeMember
ไม่กรอกชื่อรายการ	กรุณากรอกชื่อรายการค่าใช้จ่าย	Expense
จำนวนเงินว่าง / ไม่ใช่ตัวเลข / ≤ 0	กรุณากรอก... / ...ต้องเป็นตัวเลขเท่านั้น / ...ต้องมากกว่า 0	StyledTextField, Expense
ไม่เลือกผู้ร่วมหาร	กรุณาเลือกผู้ร่วมหารอย่างน้อย 1 คน	Expense
กิจกรรม: ราคาติดลบ / กรอก 0 ทุกคน	ราคาของ ... ต้องไม่ติดลบ / กรุณากรอกราคาอย่างน้อย 1 คน	ActivityExpense
วันกลับก่อนวันเดินทาง / เกิน 60 วัน	วันกลับต้องไม่ก่อนวันเดินทาง / ทริปยาวได้ไม่เกิน 60 วัน	Trip.setDates

เทคนิคด้านหน้าจอ

วาด Component เอง (Custom Painting)

ปุ่ม การ์ด ไอคอน กราฟโดนัท และแบนเนอร์ override `paintComponent()` แล้ววาดด้วย `Graphics2D` + Anti-aliasing ไม่ใช้ไฟล์รูปหรือไลบรารีภายนอก

ฟอนต์ฝังในโปรแกรม

โหลด Kanit (หัวข้อ) และ Prompt (เนื้อหา) จากไฟล์ .ttf ด้วย `Font.createFont()` แสดงผลเหมือนกันทุกเครื่องโดยไม่ต้องติดตั้งฟอนต์

Design System (Theme)

สีและฟอนต์ทั้งหมดอยู่ใน `Theme.java` แก้ที่เดียวแล้วทั้งโปรแกรมเปลี่ยนตาม

TableModel + Renderer

ตารางดึงข้อมูลจาก Trip โดยตรงผ่าน `AbstractTableModel` และใช้ Renderer แสดงไอคอน avatar และป้ายสถานะสีในเซลล์

Layout ที่ปรับตามขนาดจอ

ใช้ BorderLayout / GridBagLayout / CardLayout ถ้าจอเล็กกว่า 1440×900 จะเปิดเต็มจอ และเลื่อนหน้าได้เหมือนหน้าเว็บ

ภาษาไทยและปี พ.ศ.

`ThaiDates` แปลงวันที่เป็นภาษาไทย ปี พ.ศ. เช่น "16 – 18 ตุลาคม 2569" และตัวเลขเงินแสดงแบบ 1,234.00

9 โครงสร้างโปรเจกต์: โค้ดส่วนไหนทำอะไร

ทุกไฟล์มีกล่องคอมเมนต์ `// # ไฟล์ / ส่วน / หน้าที่ / จุดสำคัญ` ที่หัวไฟล์ และเลขส่วน `// # 1.`, `// # 2.` … กำกับทุกส่วนของโค้ด จุดที่มีเครื่องหมาย ★ คือจุดสำคัญที่ควรอธิบายตอนนำเสนอ

```
TripSplit/
├── src/tripsplit/
│   ├── Main.java, SampleData.java        จุดเริ่มต้น + ข้อมูลตัวอย่าง
│   ├── model/      (11 ไฟล์ · 880 บรรทัด)  ข้อมูล + การคำนวณ (ไม่มีโค้ดหน้าจอ)
│   └── ui/         (11 ไฟล์ · 2,526 บรรทัด) หน้าจอหลัก ฟอร์ม ตาราง ธีม
│       ├── components/ (14 ไฟล์ · 2,514 บรรทัด) Component ที่ออกแบบเอง
│       ├── fonts/  Kanit + Prompt (.ttf)    images/  ใส่ hero.jpg ได้
├── test/tripsplit/TripModelTest.java     Test Case 22 กรณี
├── TripSplit.jar · run.bat · README.md · docs/screenshots/
```

แพ็กเกจ model MODEL

ไฟล์	หน้าที่	เมธอด / ส่วนสำคัญ
Trip	ศูนย์กลางของระบบ เก็บสมาชิก ค่าใช้จ่าย วันที่ และคำนวณทุกอย่าง	`addMember` `removeMember` `addExpense` `setDates` `getTotalSpending` `getBalance` `getTotalByCategory` ★`calculateSettlements` `buildSummaryText`
Splittable (interface)	สัญญาของสิ่งที่หารเงินได้	`getTotal` `getShareFor` `involves`
Expense (abstract)	คลาสแม่ของค่าใช้จ่าย: ชื่อ ผู้จ่าย ผู้ร่วมหาร	abstract `getCategory/getTotal/getDetail`, `getShareFor` (หารเท่ากัน), `requirePositive`
FoodExpense	ค่าอาหาร + Service Charge 10%	`getTotal` = ค่าอาหาร × 1.10
TransportExpense	ค่าเดินทาง หารเท่ากัน	`getTotal` = จำนวนเงิน
HotelExpense	ค่าที่พัก	`getTotal` = ราคา × คืน × ห้อง
ActivityExpense	ค่ากิจกรรม ราคาแยกรายคน	`getTotal` = Σราคา, override `getShareFor` = ราคาของตัวเอง
Member	สมาชิก 1 คน	ตรวจชื่อใน constructor, `equals/hashCode` กันชื่อซ้ำ
Settlement	ผลการโอน 1 รายการ	`from` → `to` : `amount`
ExpenseCategory (enum)	หมวด 4 หมวด + ชื่อไทย	FOOD, TRANSPORT, HOTEL, ACTIVITY
Money	จัดรูปแบบตัวเลขเงิน	`format(1234.5)` → "1,234.50"

แพ็กเกจ ui UI

ไฟล์	หน้าที่	เมธอด / ส่วนสำคัญ
TripSplitForm (JFrame)	หน้าต่างหลัก ประกอบทุกส่วนเข้าด้วยกัน	`buildTopNav` `buildPage` `buildBody` `showView` ★ `refreshAll` `editTrip` `createTrip`
MemberPanel	การ์ดเพิ่ม/ลบสมาชิก	`addMember` `removeMember` `refresh` `createChip`
ExpenseFormPanel	ฟอร์มเพิ่มค่าใช้จ่าย	`selectCategory` `rebuildActivityRows` ★ `buildExpense` `updatePreview` `onAdd`
SummaryPanel	การ์ดสรุป 3 แท็บ	`showTab` `refreshSettlements` `settlementRow` `copySummary`
ExpenseTableModel	เชื่อมรายการค่าใช้จ่ายกับ JTable	`getValueAt` เรียก `getTotal/getDetail` (Polymorphism)
BalanceTableModel	เชื่อมยอดรายคนกับ JTable	`getTotalPaidBy` `getTotalShareOf` `getBalance`
TableRenderers	หน้าตาเซลล์ตาราง	Text / Money / Category / Member / BalanceStatus Renderer
TripDialog	หน้าต่างสร้าง/แก้ไขทริป	`showCreate` `showEdit` `onSave` (ตรวจด้วยกฎของ Model)
Theme	สี ฟอนต์ ค่าตั้งต้นทั้งโปรแกรม	`loadFont` ★ `font` `aa` `applyGlobalDefaults`
UI	ฟังก์ชันช่วยสร้าง/ตกแต่ง Component	`label` `scroll` `checkBox` `styleTable` `warn/info/confirm`
ThaiDates	วันที่ภาษาไทย ปี พ.ศ.	`format` `formatRange` `duration`

แพ็กเกจ ui.components COMPONENT

ไฟล์	หน้าที่
TopNavBar	แถบเมนูด้านบน: โลโก้ แท็บ (มีเส้นใต้เมื่อเลือก) และปุ่มด้านขวา
HeroBanner	แบนเนอร์ทริป วาดภูเขา 5 ชั้น หมอก ต้นสน และเจดีย์ด้วยโค้ด + ชื่อทริป วันที่ รูปสมาชิก
StatCard	การ์ดตัวเลขสรุป จัดเนื้อหากึ่งกลาง มีแถบสีด้านบนและบรรทัดสรุป
DonutChart	กราฟโดนัทสัดส่วนค่าใช้จ่ายแยกหมวด พร้อมคำอธิบายและแถบเปอร์เซ็นต์
DatePickerField	ช่องเลือกวันที่ + ปฏิทินรายเดือนแบบ popup (ปี พ.ศ.)
CategoryButton	ปุ่มเลือกหมวด (ไอคอน + ชื่อ) ไฮไลต์สีประจำหมวดเมื่อเลือก
ModernButton	ปุ่มแบน มุมโค้ง มี hover รองรับ 8 สไตล์ (enum Style)
StyledTextField	ช่องกรอกมุมโค้ง + placeholder และ ★ `getNumber()` ตรวจตัวเลข
Icons	ไอคอนทั้งหมดวาดด้วยโค้ด: หมวด, avatar, checkbox, เมนู, การ์ดสรุป, โลโก้
อื่น ๆ	RoundedPanel (การ์ดมุมโค้งมีเงา) · RoundedBorder (ขอบช่องกรอก) · PillLabel (ป้ายแคปซูล) · Stepper (ปุ่ม - ค่า +) · ModernScrollBarUI (แถบเลื่อนบาง)

ROOT **Main** เปิดหน้าต่างบน Event Dispatch Thread (รองรับ `--demo`) · **SampleData** ใส่ข้อมูลทริปตัวอย่าง · TEST **TripModelTest** ทดสอบ Model 22 กรณี

10 ลำดับการทำงานเมื่อกด "บันทึกค่าใช้จ่าย"

ตัวอย่างนี้แสดงว่า UI กับ Model ทำงานร่วมกันอย่างไร ตั้งแต่ผู้ใช้กดปุ่มจนหน้าจออัปเดต

1. ผู้ใช้กดบันทึก
ExpenseFormPanel `onAdd()` ตรวจว่ามีสมาชิกและชื่อรายการ

2. สร้าง Object
`buildExpense()` เลือกคลาสลูกตามหมวด เช่น `new HotelExpense(...)`

3. ส่งให้ Model
`trip.addExpense(e)` ตรวจผู้จ่าย/ผู้ร่วมหารอยู่ในทริป

4. แจ้งหน้าหลัก
เรียก callback `onChange.run()` → `refreshAll()`

5. วาดใหม่
ตาราง การ์ดสรุป แบนเนอร์ และสรุปการโอนดึงข้อมูลล่าสุดจาก Trip

```
// ExpenseFormPanel.buildExpense() — ★ จุดเชื่อม UI → Model
switch (category) {
    case FOOD:      return new FoodExpense(title, payer, participants, foodAmount.getNumber("ค่าอาหาร"), serviceCharge.isSelected());
    case TRANSPORT: return new TransportExpense(title, payer, participants, transportAmount.getNumber("ค่าเดินทาง"));
    case HOTEL:     return new HotelExpense(title, payer, participants, price, nights, rooms);
    case ACTIVITY:  return new ActivityExpense(title, payer, prices);  // ราคารายคน
}
// หลังจากนี้ทั้งโปรแกรมใช้งานผ่านชนิด Expense ตัวเดียว (Polymorphism)
```

ถ้าข้อมูลผิด: constructor ของคลาสใน Model หรือ `getNumber()` จะโยน `IllegalArgumentException` พร้อมข้อความภาษาไทย → `onAdd()` จับไว้ด้วย try-catch → แสดง `JOptionPane` "ข้อมูลไม่ถูกต้อง" และข้อมูลใน Trip ไม่ถูกเปลี่ยนแปลง

ลำดับการทำงานอื่น ๆ

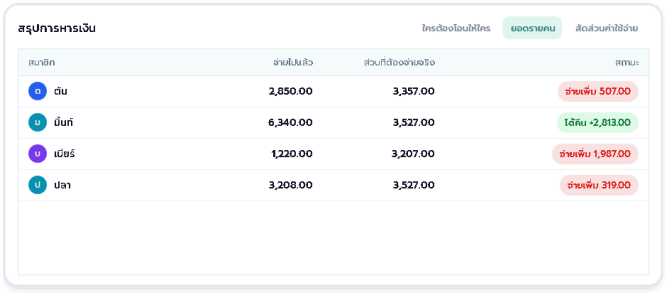
การกระทำ	สิ่งที่เกิดขึ้นในโค้ด
เพิ่มสมาชิก	`MemberPanel.addMember()` → `trip.addMember(name)` → `refreshAll()` (ฟอร์มได้รายชื่อผู้จ่าย/ผู้ร่วมหารใหม่อัตโนมัติ)
พิมพ์จำนวนเงิน	DocumentListener → `updatePreview()` สร้าง Expense ชั่วคราวเพื่อคำนวณยอดตัวอย่าง (ยังไม่บันทึก)
เปิดแท็บสรุป	`showView()` ซ่อนการ์ดตาราง ขยายการ์ดสรุป → `SummaryPanel.refresh()` → `trip.calculateSettlements()`
คัดลอกสรุป	`copySummary()` → `trip.buildSummaryText()` → ส่งข้อความเข้า Clipboard
แก้ไขทริป	คลิกชื่อทริปบนแบนเนอร์ → `TripDialog.showEdit()` → `trip.setName()`, `trip.setDates()` → `refreshAll()`

11 การทดสอบ (Test Case)

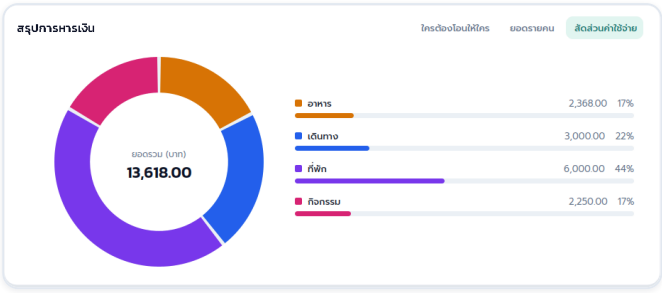
ไฟล์ `test/tripsplit/TripModelTest.java` ทดสอบ Model โดยตรงโดยไม่ต้องเปิดหน้าจอ และไม่ต้องติดตั้ง JUnit รันด้วยคำสั่ง `java -cp out tripsplit.TripModelTest` ผลการทดสอบ: **ผ่าน 22 / 22 กรณี**

รหัส	กรณีทดสอบ	กลุ่ม	ผล
TC01	อาหาร 1,000 + Service Charge 10% = 1,100	สูตรแต่ละประเภท	ผ่าน
TC02	ที่พัก 2 ห้อง × 2 คืน × 1,200 = 4,800	สูตรแต่ละประเภท	ผ่าน
TC03	กิจกรรมราคารายคน 200 + 0 + 100 = 300	สูตรแต่ละประเภท	ผ่าน
TC04	คนที่กรอก 0 ส่วนแบ่ง = 0	สูตรแต่ละประเภท	ผ่าน
TC05	แต่ละคนรับผิดชอบเท่าราคาของตัวเอง (A = 200)	สูตรแต่ละประเภท	ผ่าน
TC06	ค่าเดินทาง 900 หาร 3 = 300 ต่อคน	สูตรแต่ละประเภท	ผ่าน
TC07	ยอดรวมทั้งทริป = 7,100	ยอดรวม	ผ่าน
TC08	ผลรวมยอดสุทธิทุกคน = 0	ยอดรวม	ผ่าน
TC09	โอนตามผลลัพธ์แล้ว ทุกคนยอดเป็น 0	การโอนเงิน	ผ่าน
TC10	จำนวนครั้งที่โอน ≤ จำนวนสมาชิก - 1	การโอนเงิน	ผ่าน
TC11	เพิ่มชื่อซ้ำไม่ได้	ตรวจข้อมูล	ผ่าน
TC12	ชื่อว่างไม่ได้	ตรวจข้อมูล	ผ่าน
TC13	จำนวนเงินต้องมากกว่า 0	ตรวจข้อมูล	ผ่าน
TC14	ต้องมีผู้ร่วมหารอย่างน้อย 1 คน	ตรวจข้อมูล	ผ่าน
TC15	ลบสมาชิกที่มีรายการเกี่ยวข้องไม่ได้	ตรวจข้อมูล	ผ่าน
TC16	ลบสมาชิกที่ไม่มีรายการได้	ตรวจข้อมูล	ผ่าน
TC17	หาร 100 บาท 3 คน: เพื่อนโอนคืนรวม 66.66	ทศนิยม	ผ่าน
TC18	กิจกรรมกรอก 0 ทุกคนไม่ได้	ตรวจข้อมูล	ผ่าน
TC19	ราคาติดลบไม่ได้	ตรวจข้อมูล	ผ่าน
TC20	ทริป 16–18 ต.ค. = 3 วัน	วันเดินทาง	ผ่าน
TC21	ทริป 3 วัน = 2 คืน	วันเดินทาง	ผ่าน
TC22	วันกลับก่อนวันเดินทางไม่ได้	วันเดินทาง	ผ่าน

12 วิธีรันโปรแกรมและสรุป



แท็บยอดรายคน: จ่ายไปแล้ว / ส่วนที่ต้องจ่าย / สถานะ



แท็บสัดส่วนค่าใช้จ่าย (กราฟโดนัท)

วิธีรัน (ต้องมี JDK 17 ขึ้นไป)

วิธี	ขั้นตอน
ไฟล์ JAR	ดับเบิลคลิก `TripSplit.jar` หรือสั่ง `java -jar TripSplit.jar`
Windows	ดับเบิลคลิก `run.bat` (คอมไพล์และรันให้อัตโนมัติ)
IntelliJ IDEA	Open โฟลเดอร์ TripSplit → ตั้ง `src` เป็น Sources Root → ตั้ง Encoding เป็น UTF-8 → รัน `Main.java`
Command line	`javac -encoding UTF-8 -d out ...` แล้ว `java -cp out tripsplit.Main --demo` (คัดลอกโฟลเดอร์ fonts ไปที่ out ด้วย)

สรุป

Trip Split ช่วยให้กลุ่มเพื่อนบันทึกค่าใช้จ่ายระหว่างทริป และรู้ทันทีว่าใครต้องโอนให้ใครด้วยจำนวนครั้งน้อยที่สุด ตอบโจทย์ Workshop ครบทุกข้อ:

- **GUI ด้วย Java Swing**: หน้าต่างเดียวแบบ Dashboard, Component ที่ออกแบบเอง, ภาษาไทยทั้งหมด
- **แยก Model และ UI**: Logic ทั้งหมดอยู่ใน `model` ทดสอบได้โดยไม่ต้องเปิดหน้าจอ
- **OOP ครบ 4 ข้อ**: Encapsulation, Inheritance, Polymorphism, Abstraction และความสัมพันธ์ Has-a / Has-many
- **ข้อมูลหลาย Class**: Trip มี Member และ Expense หลายตัว, Expense มีคลาสลูก 4 แบบ
- **ตรวจสอบข้อมูล**: แจ้งเตือนด้วย JOptionPane และมี Test Case 22 กรณีผ่านทั้งหมด